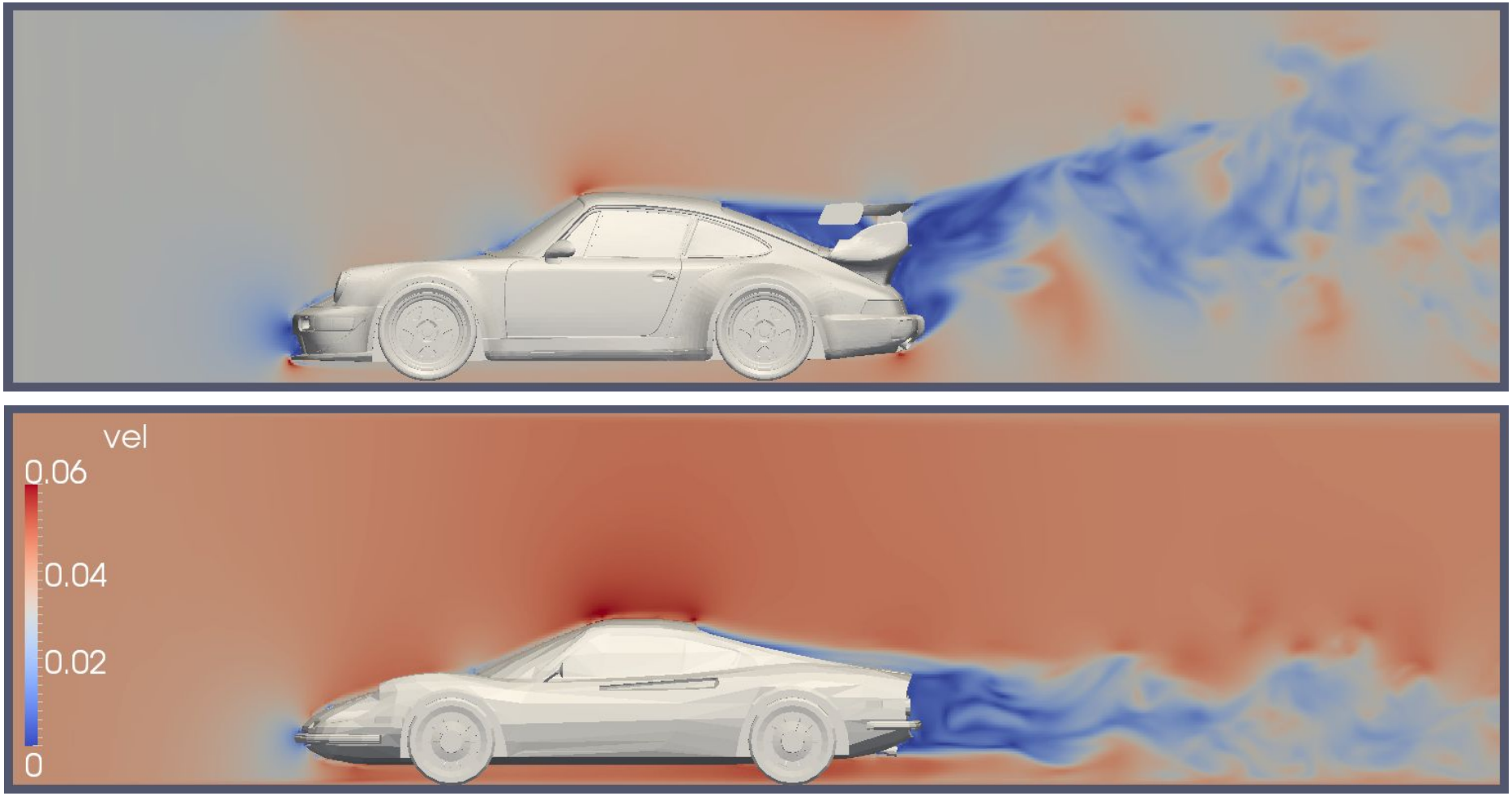
The wall reflection method and wall function method for D3Q15 Lattice Boltzmann Method

Giles Richardson (GARCFD)

17th March 2025


vel
0.06
0.04
0.02
0

Required arrays

The STL normals will be stored as **nrm(ntri,3)** when the STL file is read in.

The STL number (1 to nstl) will be stored at each cutcell in the domain **cellnrm(i,j,k).**

(So that we know which STL triangle and normal are associated with each cutcell)

The cutcell number **cutcell(i,j,k)** will also be stored at each cell - this is just an incremental number (1:ncutc) which will later be used to access the (real) **weight** and (integer) **normal** arrays. Since we dont loop through the cutcells separately, we need a safe method to refer to each cutcell number, especially when running in parallel.

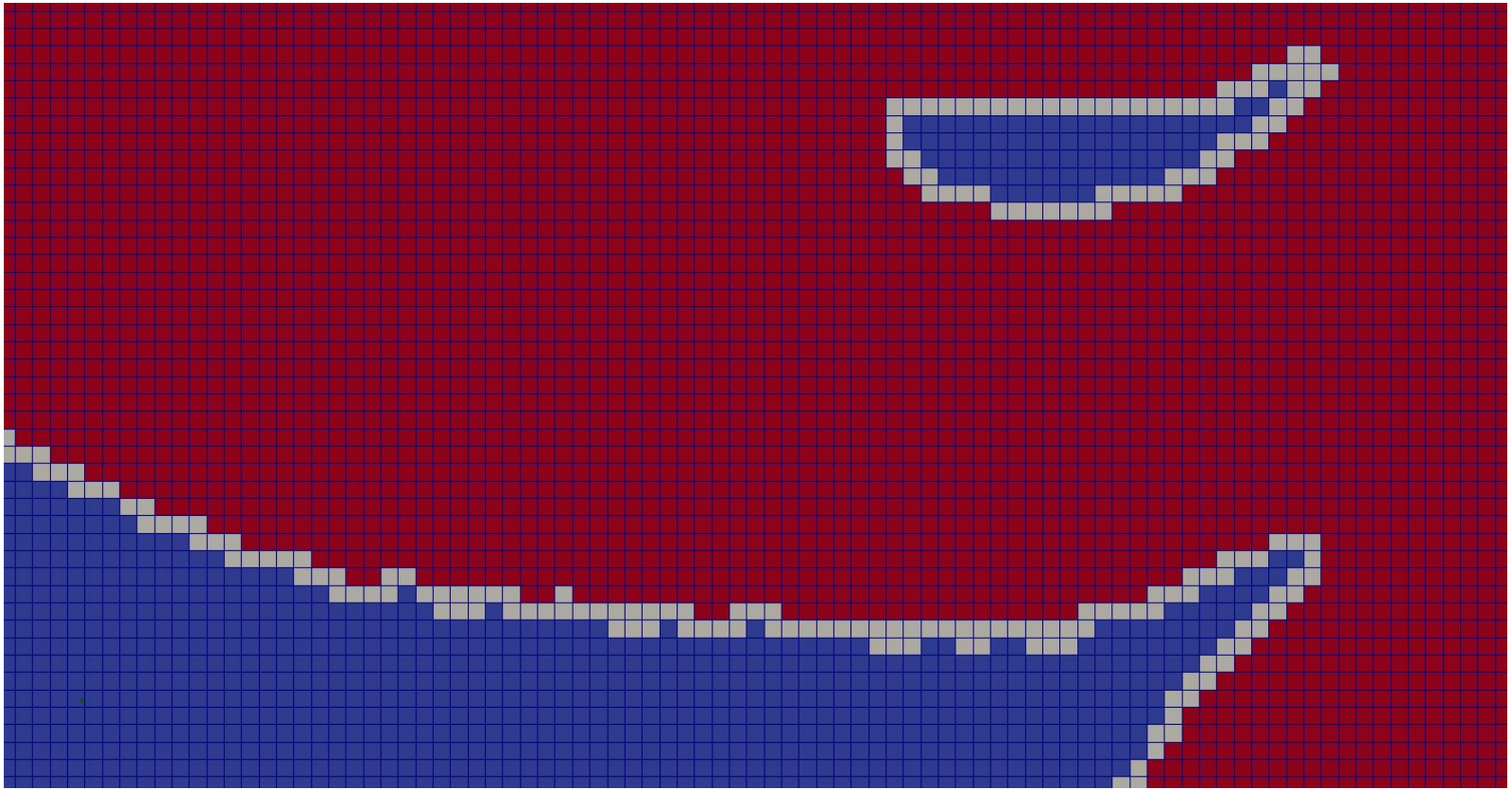
background information

The mesh used is a uniform cartesian mesh. The STL geometry is used to mark cutcells. Then a floodfill method is used to fill all the live cells. The rest of the cells remain as dead cells. For convenience this is referred to as the **lcd** array (live/cut/dead).

The LBM method is only applied to the live and cutcells. The dead cells are ignored. The cutcells must each have at least one live cell neighbour, otherwise they will be reassigned as a dead cell.

Whether using the bounce-back method or reflection method, it will only be applied to the cutcells, not the live or dead cells.

cutcells (grey), live cells (red), dead cells (blue)



The 15 direction vectors and weights are defined as:

```
vec(1, :) = (/ 1,-1,-1 /)
vec(2, :) = (/ 1,-1, 1 /)
vec(3, :) = (/ 1, 0, 0 /)
vec(4, :) = (/ 1, 1,-1 /)
vec(5, :) = (/ 1, 1, 1 /)
vec(6, :) = (/ 0,-1, 0 /)
vec(7, :) = (/ 0, 0,-1 /)
vec(8, :) = (/ 0, 0, 0 /)
vec(9, :) = (/ 0, 0, 1 /)
vec(10,:) = (/ 0, 1, 0 /)
vec(11,:) = (/-1,-1,-1 /)
vec(12,:) = (/-1,-1, 1 /)
vec(13,:) = (/-1, 0, 0 /)
vec(14,:) = (/-1, 1,-1 /)
vec(15,:) = (/-1, 1, 1 /)
```

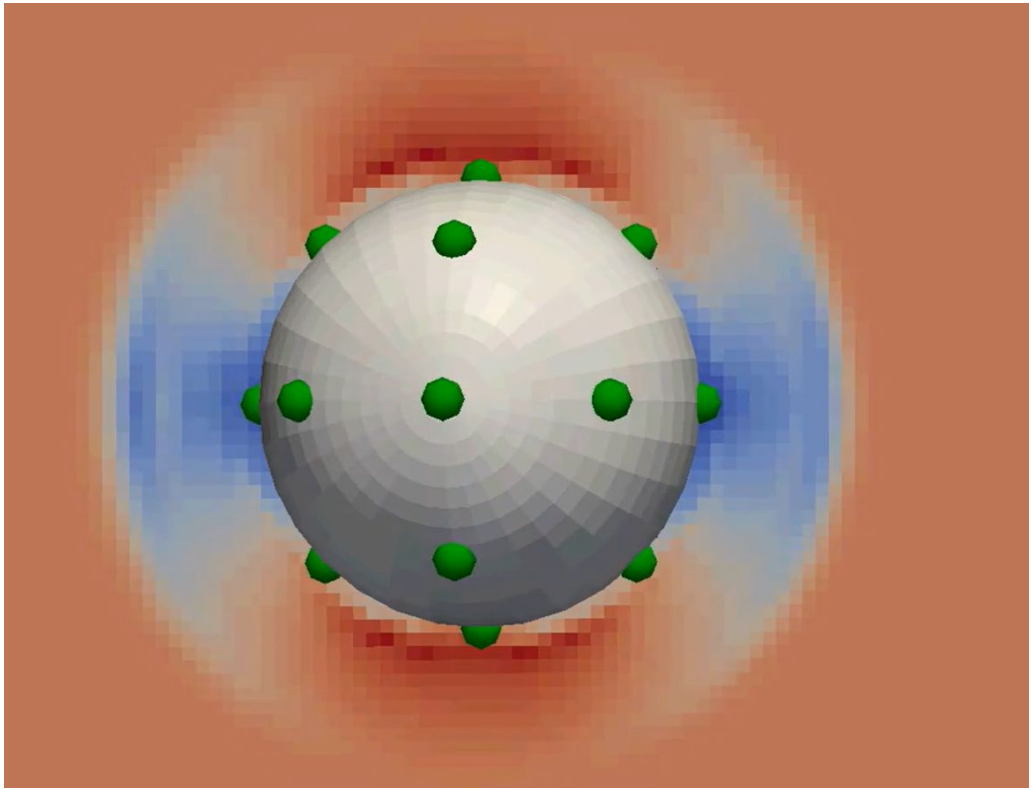
```
wt(1)  = 1./72.
wt(2)  = 1./72.
wt(3)  = 1./9.
wt(4)  = 1./72.
wt(5)  = 1./72.
wt(6)  = 1./9.
wt(7)  = 1./9.
wt(8)  = 2./9.
wt(9)  = 1./9.
wt(10) = 1./9.
wt(11) = 1./72.
wt(12) = 1./72.
wt(13) = 1./9.
wt(14) = 1./72.
wt(15) = 1./72.
```

The 18 ‘discrete’ normals are defined as:

```
nnrm = 18

nrm18(1, :) = [-0.70710676,-0.70710676, 0.00000000]
nrm18(2, :) = [-0.70710676, 0.00000000,-0.70710676]
nrm18(3, :) = [-1.00000000, 0.00000000, 0.00000000]
nrm18(4, :) = [-0.70710676, 0.00000000, 0.70710676]
nrm18(5, :) = [-0.70710676, 0.70710676, 0.00000000]
nrm18(6, :) = [ 0.00000000,-0.70710676,-0.70710676]
nrm18(7, :) = [ 0.00000000,-1.00000000, 0.00000000]
nrm18(8, :) = [ 0.00000000,-0.70710676, 0.70710676]
nrm18(9, :) = [ 0.00000000, 0.00000000,-1.00000000]
nrm18(10,:) = [ 0.00000000, 0.00000000, 1.00000000]
nrm18(11,:) = [ 0.00000000, 0.70710676,-0.70710676]
nrm18(12,:) = [ 0.00000000, 1.00000000, 0.00000000]
nrm18(13,:) = [ 0.00000000, 0.70710676, 0.70710676]
nrm18(14,:) = [ 0.70710676,-0.70710676, 0.00000000]
nrm18(15,:) = [ 0.70710676, 0.00000000,-0.70710676]
nrm18(16,:) = [ 1.00000000, 0.00000000, 0.00000000]
nrm18(17,:) = [ 0.70710676, 0.00000000, 0.70710676]
nrm18(18,:) = [ 0.70710676, 0.70710676, 0.00000000]
```

The 18 discrete normals look like this, where the green points represent the normal vector endpoints on a sphere



The reflection matrix is defined as follows:
(each row corresponds to each discrete normal)
Note: Rows 1-9 are mirrored in rows 10-18

```
refl8(1, :) = [ 1, 2, 6,11,12, 3, 7, 8, 9,13, 4, 5,10,14,15]
refl8(2, :) = [ 1,11, 7, 4,14, 6, 3, 8,13,10, 2,12, 9, 5,15]
refl8(3, :) = [11,12,13,14,15, 6, 7, 8, 9,10, 1, 2, 3, 4, 5]
refl8(4, :) = [12, 2, 9,15, 5, 6,13, 8, 3,10,11, 1, 7,14, 4]
refl8(5, :) = [14,15,10, 4, 5,13, 7, 8, 9, 3,11,12, 6, 1, 2]
refl8(6, :) = [ 5, 2, 3, 4, 1, 9,10, 8, 6, 7,15,12,13,14,11]
refl8(7, :) = [ 4, 5, 3, 1, 2,10, 7, 8, 9, 6,14,15,13,11,12]
refl8(8, :) = [ 1, 4, 3, 2, 5, 7, 6, 8,10, 9,11,14,13,12,15]
refl8(9, :) = [ 2, 1, 3, 5, 4, 6, 9, 8, 7,10,12,11,13,15,14]
refl8(10,:) = [ 2, 1, 3, 5, 4, 6, 9, 8, 7,10,12,11,13,15,14]
refl8(11,:) = [ 1, 4, 3, 2, 5, 7, 6, 8,10, 9,11,14,13,12,15]
refl8(12,:) = [ 4, 5, 3, 1, 2,10, 7, 8, 9, 6,14,15,13,11,12]
refl8(13,:) = [ 5, 2, 3, 4, 1, 9,10, 8, 6, 7,15,12,13,14,11]
refl8(14,:) = [14,15,10, 4, 5,13, 7, 8, 9, 3,11,12, 6, 1, 2]
refl8(15,:) = [12, 2, 9,15, 5, 6,13, 8, 3,10,11, 1, 7,14, 4]
refl8(16,:) = [11,12,13,14,15, 6, 7, 8, 9,10, 1, 2, 3, 4, 5]
refl8(17,:) = [ 1,11, 7, 4,14, 6, 3, 8,13,10, 2,12, 9, 5,15]
refl8(18,:) = [ 1, 2, 6,11,12, 3, 7, 8, 9,13, 4, 5,10,14,15]
```

The reflection matrix looks complicated but it’s been calculated by a simple fortran code, which looks at all the incoming vectors, and reflects them depending on the surface normal.

Many surface normals were considered, and thus we are able to focus only on the surface normals which give proper reflection of the incoming vectors to the outgoing vectors.

This saves having to recalculate the reflections during the flow solver.

Calculating the **nearest normals** and **normal weights** at each cutcell

Before the solver starts we need to loop through all the cutcells and calculate the nearest 3 discrete normals and the corresponding weight for each discrete normal.

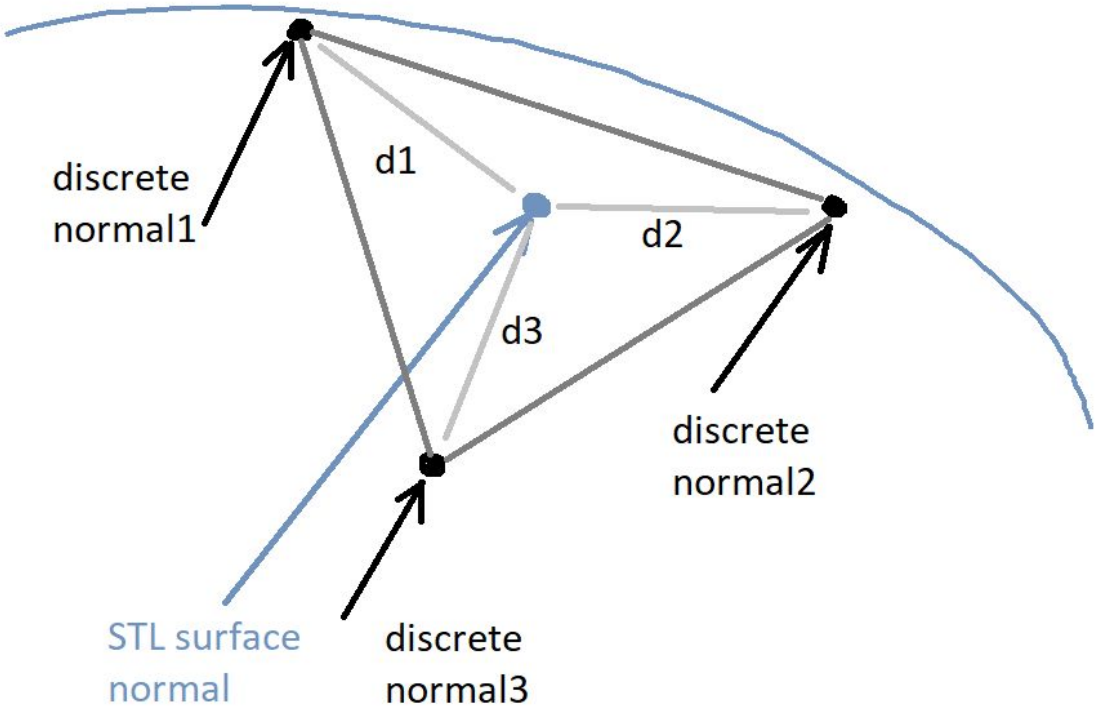
They will be stored as: **weights(ncutc,3)** and **normals(ncutc,3)**

First of all we calculate the nearest 3 discrete normals for each surface normal.

Then we store these 3 discrete normals as 3 integers (1-18) at each cutcell.

Then by calculating the distances d1,d2,d3 (from the surface normal to each discrete normal) we can assign a weight to each of the 3 discrete normals:

Calculating the distance to each discrete normals



Calculating the weights for each discrete normal

First we define a min edge length mel = 0.7654. This is roughly equivalent to the distance between closest discrete normal endpoints. Or pi/4 (chord length).

By this method we end up with 3 weights which sum to 1.0, for each cutcell.

Each weight corresponds to one of the 3 nearest discrete normals

```
mel = 0.7654
ww1 = 1.0 - min(mel,dd1)/mel
ww2 = 1.0 - min(mel,dd2)/mel
ww3 = 1.0 - min(mel,dd3)/mel

mag = ww1 + ww2 + ww3
ww1 = ww1/mag
ww2 = ww2/mag
ww3 = ww3/mag
```

How to implement the wall reflection method in practice

Having done the groundwork in calculating the nearest normals and the weights, the wall reflection method can then be implemented as follows:

At each cutcell we look at the nearest discrete normals, and use those to calculate the reflection vectors for each of the discrete normals eg **ref1 = ref18(nn1,n)**, for all 15 vectors.

Then we use the weights applied to **fin(i,j,k)** which is then added to **fout(i,j,k)** for each of the reflected directions.

So to summarise, at each cutcell we use the 3 **normals** and 3 **weights**, to calculate the proportion of **fin** that is split 3 ways by adding to **fout** (for each of the the 3 reflection directions).

Coding the wall reflection method at each cutcell

```
cutc = cutcell(i,j,k)
nn1 = normals(cutc,1)
nn2 = normals(cutc,2)
nn3 = normals(cutc,3)
ww1 = weights(cutc,1)
ww2 = weights(cutc,2)
ww3 = weights(cutc,3)

fout(i,j,k,:) = 0.0

do n = 1,nvec
  ref1 = ref18(nn1,n)
  ref2 = ref18(nn2,n)
  ref3 = ref18(nn3,n)
  fout(i,j,k,ref1) = fout(i,j,k,ref1) + ww1*fin(i,j,k,n)
  fout(i,j,k,ref2) = fout(i,j,k,ref2) + ww2*fin(i,j,k,n)
  fout(i,j,k,ref3) = fout(i,j,k,ref3) + ww3*fin(i,j,k,n)
enddo
```

summary of arrays needed

stl(ntri,3,3)	STL points
nrm(ntri,3)	STL normals
cutcell(ni,nj,nk)	cutcell numbering system (1:ncutc)
cellnrm(ni,nj,nk)	cutcell STL triangle number (1:ntri)
ref18(18,15)	reflection matrix
nrm18(18,3)	discrete normals
normals(ncutc,3)	the 3 nearest discrete normals at each cutcell
weights(ncutc,3)	the 3 weights (for each discrete normal) at each cutcell

blank slide

The wall function method

Not really a wall function - but the wall reflection method by itself will still have quite low values of velocity at the wall surface, so me may want to increase the wall velocity somewhat, like you would do by using a wall function.

By default (if you set the **reach** or **wlfn** value to zero) at each cutcell it looks for the nearest live cells, and takes the average of the **fin** values (for each vector).

In other words, it loops through all 27 neighbour cells (3x3x3) and looks for the live cells only. From those live cells, it takes the sum of the **fin** values (at each vector), and divides by the number of live neighbour cells. This gives you a new fin(n) array which can be applied at the cutcell, giving you a higher wall velocity.

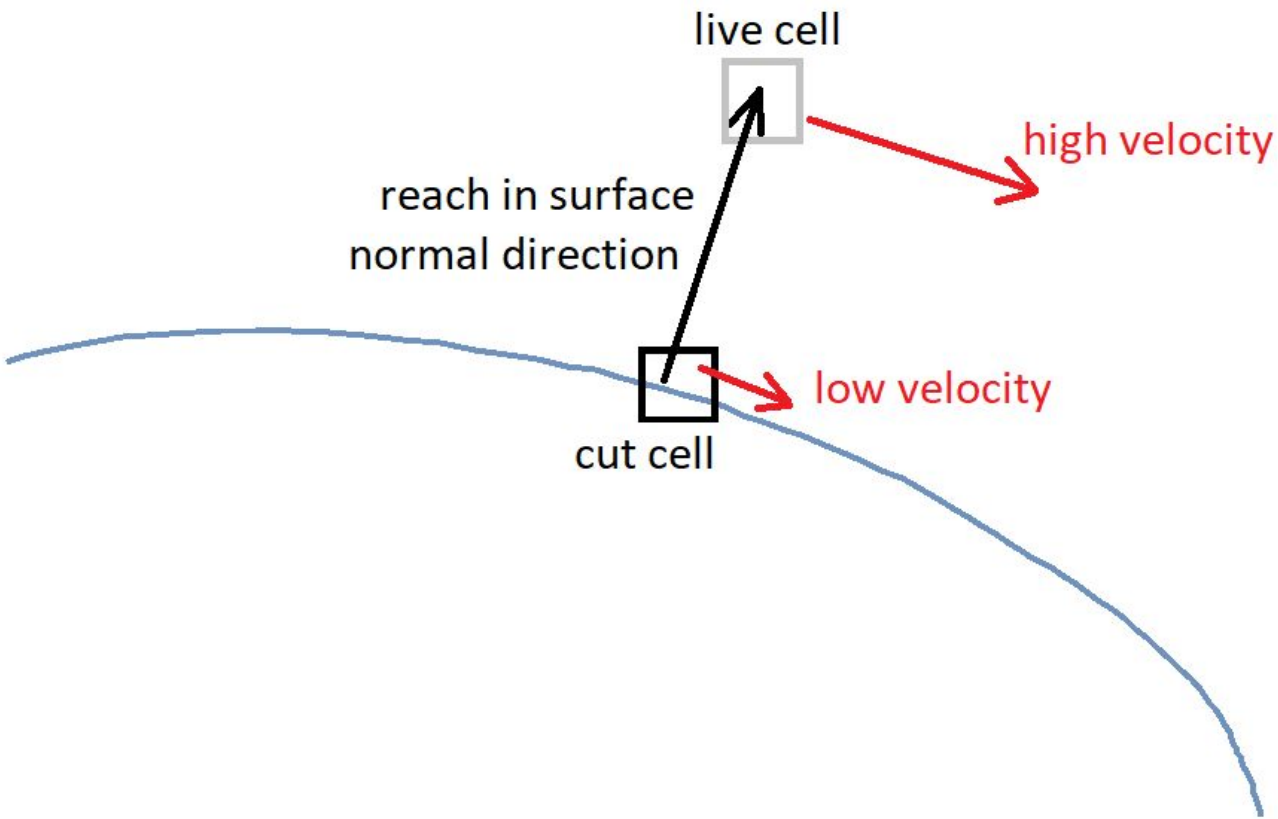
This gives a robust method for increasing the cutcell wall velocity significantly.

The wall function method

But (to improve this method) instead of taking the average of the nearest live cells (to calculate fin(n) at the cutcells), we can instead “reach” out into the flowfield and grab the fin(n) values and apply those at the cutcell instead. This will give us a much higher velocity at the wall, depending on how far out (how many cells) we reach.

Admittedly - its a crude method but it works - and allows the user to decide how far (or how many cells) to reach outwards into the flowfield. By increasing the wall velocity, we can have much better control over the location and extent of separation regions.

The wall function method…



The wall function method…

Here we assign a “reach” value (say 4 cells) to the wlfn variable. This allows us to reach out in the local surface normal direction.

The local value of fin can then be updated by using the fin values from further away from the wall, as follows:

```
nrm1 = nrm(cellnrm(i,j,k),1)
nrm2 = nrm(cellnrm(i,j,k),2)
nrm3 = nrm(cellnrm(i,j,k),3)

ii = i + nint(nrm1*real(wlfn))
jj = j + nint(nrm2*real(wlfn))
kk = k + nint(nrm3*real(wlfn))
```

```
do n = 1,nvec
  fin(i,j,k,n) = fin(ii,jj,kk,n)
enddo
```

The wall function method…

When **reaching** out into the flowfield to look for a live cell, if the geometry is messy, we might land on a dead cell or cutcell by mistake.

So in this case, we revert to the “safe” method of using the average of the nearest live cell neighbours of the cutcell.

So there is a condition in the code which says that **IF** (reaching out does not give a live cell) **THEN** (use the average of the local live cell neighbours).